

Employee Payroll Management System

PROJECT REPORT

Project Name	Employee Payroll Management System
Course	Programming in Java
Academic Term	Fall Semester 2026–27
Language & Tools	Java (JDK), Command-Line Interface, CSV File Storage
Submitted By	Sanidhya Godiyal
Registration No.	25BAI10158
Degree Program	B.Tech CSE (AIML)
School	School of Computer Science and Artificial Intelligence
University	VIT Bhopal

Index

Title	Page No.
Introduction	3
Problem Statement & Objectives	4
Functional Requirements	5
Non-Functional Requirements	6
System Architecture	7
Design Diagrams	8
Use Case Diagram	9
Workflow Diagram	10
Sequence Diagram	11
Class Diagram	12
ER / Storage Diagram	13
Design Decisions & Rationale	14
Implementation Details	15
Screenshots and results	16
Testing Approach	18
Challenges faced	19
Learnings & key takeaway	20
Future Enhancements	21
References	22

1. Introduction

Employee payroll management is a classic example that is used for learning programming, files handling while also being of importance in real industry applications. For this assignment I have developed an easy console (terminal based) application on Java that will allow user to enter, modify and delete employee details and automatically calculate the net pay of the employees. No database or GUI is used and the application is console based only.

2. Problem Statement & Objectives

Problem Statement

The intention is to create a functional payroll system that implements the basic functions of creating, retrieving, modifying, and deleting information related to employees, and keeps the code easy enough for me to explain its working and flow.

This project is a perfect mid point between a project that can be used to get familiarised with the JAVA language and somethings that actually used in day to day professional life.

Objectives

- Let a user add a new employee along with their basic details : name, designation, basic salary, allowances and deductions.
- Automatically calculate net salary instead of expecting the user to work it out by hand.
- Let the user view every employee's record in one place, in a readable format.
- Allow updating a single employee's details without needing to re-enter every field.
- Allow removing an employee's record using their ID.
- Let the user search for one specific employee quickly by ID.
- Make sure employee data is still there the next time the program is started.
- Keep the whole thing runnable from a terminal, with nothing to install beyond Java itself.

3. Functional Requirements

These are the things the programme needs to do.

- The system shall allow adding a new employee record with a name, designation, basic salary, allowances and deductions, and shall generate the employee's ID automatically.
- The system shall calculate and display each employee's net salary as basic salary + allowances – deductions.
- The system shall allow viewing the full list of employees currently stored in the system.
- The system shall allow updating an existing employee's details by ID, and shall leave a field unchanged if the user chooses not to update it.
- The system shall allow removing an employee record by ID.
- The system shall allow searching for a specific employee using their ID.
- The system shall persist all employee records to a file so the data is available the next time the application is run.
- The system shall reject invalid menu choices and invalid numeric input without crashing, and shall prompt the user to try again.

4. Non Functional Requirements

Lets see how the program behaves while doing it.

- Usability : the menu is numbered and self-explanatory, so a new user shouldn't need a manual to figure out how to add or remove an employee.
- Portability : the program only needs a Java Runtime Environment installed, it behaves the same way on Windows, macOS or Linux.
- Maintainability : the code is split into three classes with clearly separate jobs, so a change to how data is stored doesn't mean touching the menu code, and vice versa.
- Performance : for the scale this project is meant for (tens to a few hundred records), rewriting the whole CSV file on every change is fast enough that there's no noticeable delay.

5. System Architecture

The architecture is simple as there are only three JAVA classes. The user interacts with the program only through the terminal. `Main.java` reads what they type and calls the right method. `PayrollSystem.java` holds the actual list of employees in memory and knows how to add, update, remove and search through them. Whenever that list changes, `PayrollSystem` writes the whole thing back out to `employees.csv`

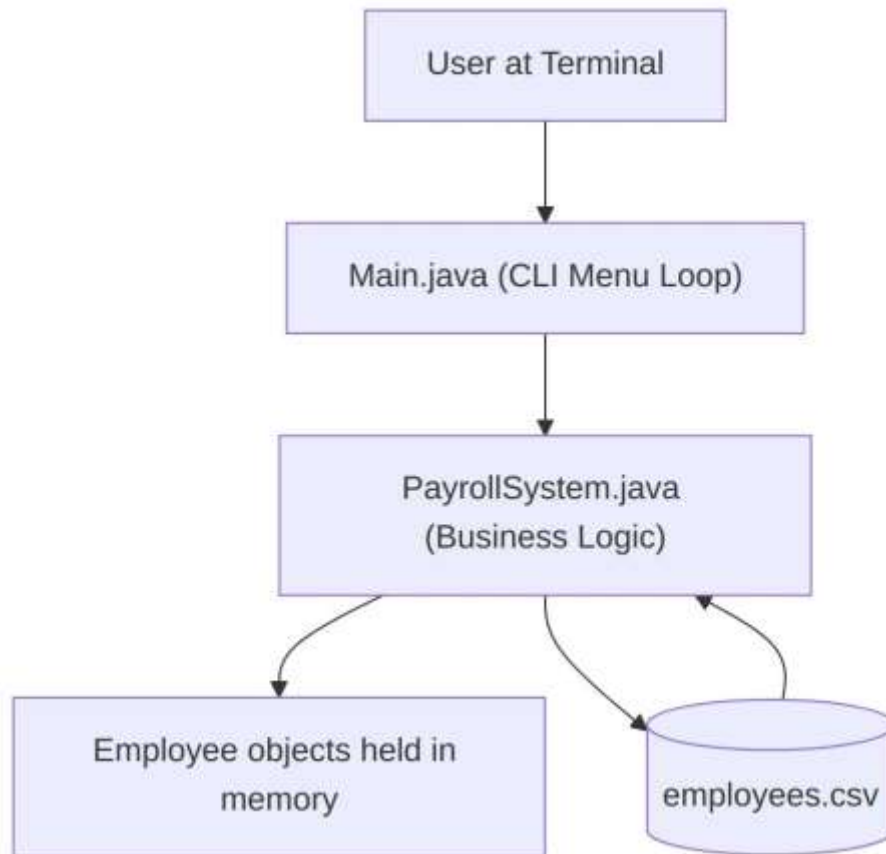


Figure 1: High-level system architecture.

The CSV file as the only thing that is common between runs, there's no database to manage. That keeps the number of moving parts small.

6. Design Diagrams

6.1 Use Case Diagram

There's really only one type of actor in this system — the person sitting at the terminal — and six things they can do. I didn't add separate actors for things like "admin" or "employee" because this project doesn't have logins or roles; whoever runs the program can do all six actions.

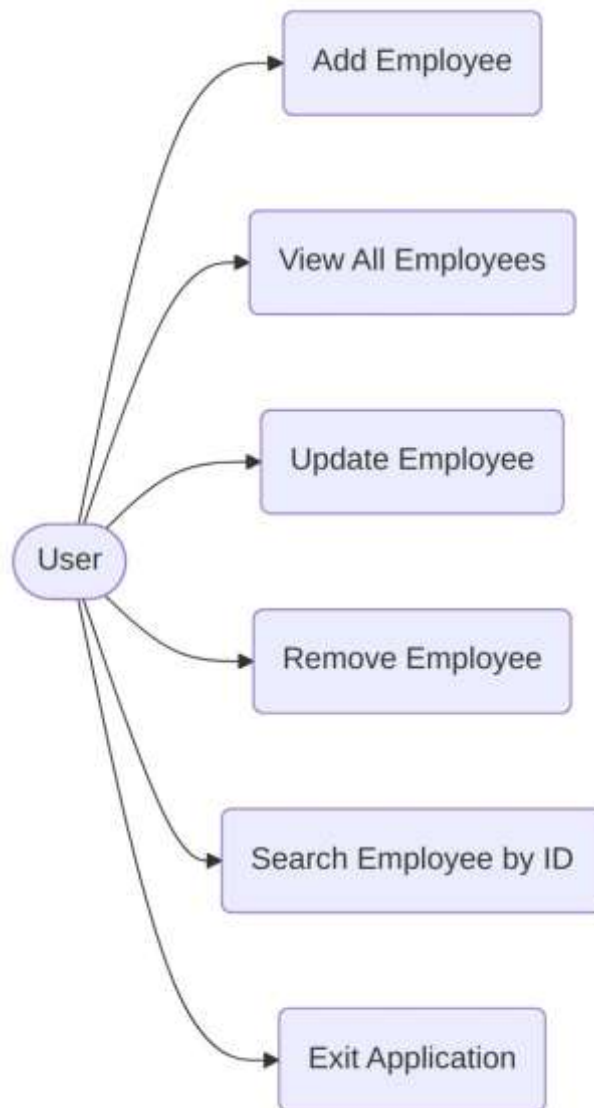


Figure 2: Use case diagram — the User actor and the six actions available from the main menu.

6.2 Workflow Diagram

This is essentially the flow of the loop in Main.java.

The program shows the menu, reads a choice, checks whether it's valid, performs the matching action, saves to the CSV file, and then either loops back to the menu or exits.

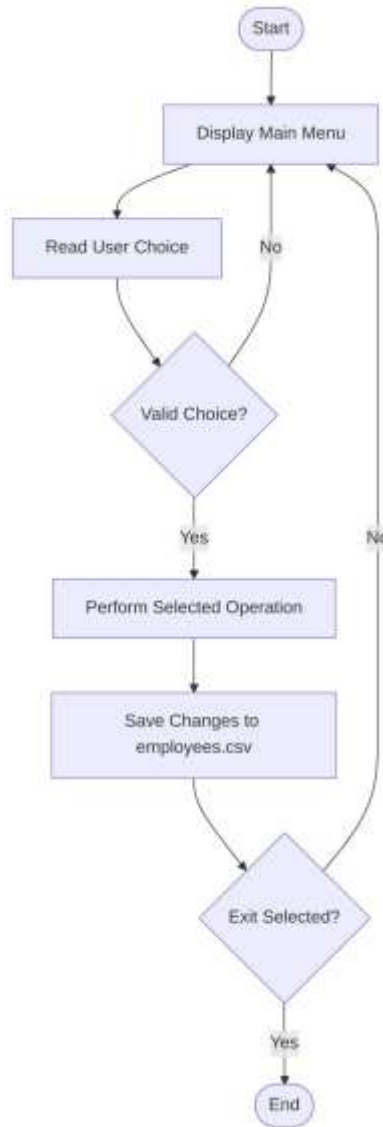


Figure 3: Workflow of the main program loop.

6.3 Sequence Diagram

6.4 Class Diagram

Three classes, one job each. Employee only stores data about a single employee and knows how to calculate its own net salary and convert itself to and from a CSV line. PayrollSystem owns the list of employees and every operation on that list, plus the file reading and writing. Main only deals with the console — printing the menu, reading input, and calling PayrollSystem. The arrows show that Main uses PayrollSystem, and PayrollSystem manages Employee objects; Employee itself doesn't depend on either of the other two classes, which is exactly what let me test it on its own before wiring up the rest.

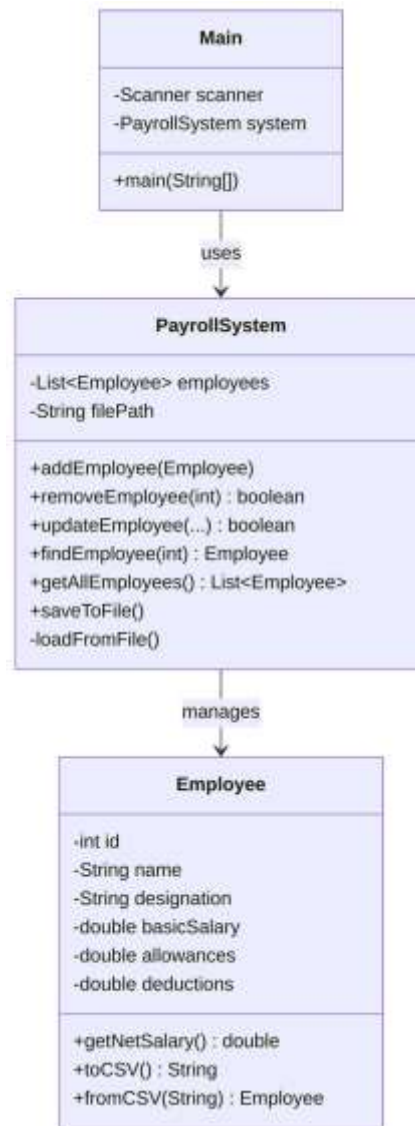


Figure 5: Class diagram showing Employee, PayrollSystem and Main.

6.5 ER / Storage Diagram

There's no relational database in this project, so a traditional ER diagram doesn't fully apply — but it's still useful to show what a single "record" looks like, since that's exactly what one row of `employees.csv` contains. I've drawn it as a single entity with its six attributes rather than forcing in a relationship that doesn't exist.

EMPLOYEE	
int	id
string	name
string	designation
double	basicSalary
double	allowances
double	deductions

Figure 6: Structure of a single employee record as stored in `employees.csv`.

7. Design Decisions & Rationale

Before writing any code I worked out what a single employee record actually needs: an ID, a name, a designation, a basic salary, allowances and deductions. The net salary can always be worked out from those (basic + allowances – deductions).

I split the program into three classes, each with one role, mainly because i had a tough experience in an earlier practice exercise where everything lived in one giant class with a main method, and debugging it was impossible for me. Employee holds the data for one employee. PayrollSystem keeps the list of all employees in memory, and knows how to add, remove, update and search them, plus how to load and save that list to a CSV file.

I used a CSV file instead of a real database mainly because it meant I didn't have to install or configure anything extra like MySQL just to run it. For a project of this size, a CSV file that gets rewritten whenever something changes is more than enough, and it also means anyone marking the project can open the data file in a spreadsheet program and see exactly what's stored, without needing any database client.

Rewriting the entire CSV file on every change, instead of trying to edit it in place, was a deliberate trade-off. It's not the most efficient approach if there were thousands of records, but it guarantees the file on disk always matches exactly what's in memory, and it removes an entire category of bugs around partial writes or leftover lines from a previous version of a record.

8. Implementation Details

Employee.java stores the six fields and has a method to work out the net salary, along with two small helper methods — toCSV and fromCSV — that convert an employee to and from a single line in the data file.

```
public double getNetSalary() {
    return basicSalary + allowances - deductions;
}
```

PayrollSystem.java loads the CSV file into a list when the program starts, and writes the entire list back to the file after every add, update or remove. The next-ID logic simply scans the current list for the highest ID in use:

```
public int getNextId() {
    int max = 0;
    for (Employee e : employees) {
        if (e.getId() > max) max = e.getId();
    }
    return max + 1;
}
```

Main.java runs a loop that prints a numbered menu and reads the user's choice using a Scanner. Each option asks for the required inputs and passes them on to PayrollSystem. The input-reading methods have some basic checks built in — for example, they ask again if the user types letters where a number is expected — so the program doesn't crash on bad input, which happened a lot while I was testing early versions of this.

All three files live in a single src folder with no packages, since a package structure felt like unnecessary ceremony for a project this size. The program is compiled with a single javac command and run with a single java command, with no build tool involved.

9. Screenshots & Results

This section is meant to show the program actually running, the main menu on startup, and the output of each of the five operations. I'm leaving marked placeholders below for now and will drop the real terminal screenshots in once I've captured them; the captions describe exactly what each one should show.

```
PS C:\Users\sanid\Downloads\employee-payroll-system> & 'C:\Program Files\Java\jdk-6.0de6098fdfdf8c7cf1aa2\redhat.java\jdt_ws\employee-payroll-system\Employee Payroll Management System
```

```
--- Main Menu ---
1. Add Employee
2. View All Employees
3. Update Employee
4. Remove Employee
5. Search Employee by ID
6. Exit
Enter your choice:
```

```
--- Add New Employee ---
Assigned Employee ID: 2
Enter name: Ram
Enter designation: CTO
Enter basic salary: 500000
Enter allowances: 100000
Enter deductions: 50000
Employee added successfully!
```

```
--- Main Menu ---
1. Add Employee
2. View All Employees
3. Update Employee
4. Remove Employee
5. Search Employee by ID
6. Exit
Enter your choice: █
```

```

--- Main Menu ---
1. Add Employee
2. View All Employees
3. Update Employee
4. Remove Employee
5. Search Employee by ID
6. Exit
Enter your choice: 2

--- All Employees ---
ID: 1 | Name: Sanidhya Godiyal | Designation: CEO | Basic: 800000.00 | Allowances: 200000.00 | Deduct
ions: 100000.00 | Net Salary: 900000.00
ID: 2 | Name: Ram | Designation: CTO | Basic: 500000.00 | Allowances: 100000.00 | Deduct
ions: 50000.00 | Net Salary: 550000.00

```

10. Testing Approach

There aren't any automated tests for this, so I recommend testing it by hand like this:

1. **Add** a couple of employees with different salaries and check if the math looks right.
2. **View** the list to make sure they show up.
3. **Update** someone, but try skipping a few fields to make sure it only changes what you want it to.
4. **Search** for a fake ID to make sure it gives you a "not found" error.
5. Try typing some random letters when it asks for a salary to make sure it catches the error and asks again instead of crashing.

11. Challenges Faced

The main issue I ran into was with the update feature. In my first version, if the user pressed Enter without typing anything, the program treated that as “set this field to blank,” which is wrong, pressing Enter should mean “leave it as it is.” I fixed this by making the input functions return null when nothing is typed, and only updating a field in PayrollSystem when the value passed in isn't null.

Another problem was number parsing. `Integer.parseInt` and `Double.parseDouble` throw an exception on invalid text, so the first version of the program would simply crash if you typed a letter into the salary field by mistake. I fixed this by wrapping those reads in a loop with a try/catch, so it just asks again instead of stopping the whole program.

A smaller but slightly annoying issue was formatting the console output so the employee list actually lined up in columns. `String.format` with fixed-width specifiers ended up being the fix, but I went through a few ugly versions where longer names pushed everything else out of alignment before I got the widths right.

12. Learnings & Key Takeaways

This project was the first time splitting a program into separate classes actually felt useful to me rather than like an exercise I was told to do. When something went wrong with saving data, I knew immediately to look in `PayrollSystem` and not in `Main`, and that alone saved me a lot of time compared to earlier assignments where everything lived in one file.

I also got a lot more comfortable with basic file handling in Java, `BufferedReader`, `PrintWriter`, and the slightly annoying reality that reading a file that doesn't exist yet needs to be handled gracefully rather than assumed away. Wrapping user input in loops with `try/catch` instead of trusting that people will always type what you expect was another habit I picked up directly from watching this program crash on my own bad test input.

Probably the biggest takeaway, though, is a bit less technical: sketching out the classes and their responsibilities on paper before writing code saved me from a messy rewrite halfway through. It's tempting to just start typing, especially under a deadline, but the twenty minutes I spent planning the three-class split more than paid for itself later.

13. Future Enhancements

- Move from a CSV file to a lightweight embedded database such as SQLite, so the storage layer can handle far more records without rewriting the whole file on every change.
- Add basic unit tests with JUnit for the PayrollSystem methods, instead of relying entirely on manual testing.
- Generate a simple payslip for each employee, either as a plain text file or a PDF.
- Add basic tax or statutory deduction calculations instead of a single generic “deductions” field.
- Export the employee list to an Excel (.xlsx) file for anyone who wants to work with the data outside the program.
- Optionally add a simple JavaFX front end for anyone who'd prefer a graphical interface over the terminal menu, while keeping the CLI version as the primary mode.

14. References

1. Oracle. Java Platform, Standard Edition Documentation. <https://docs.oracle.com/en/java/javase/>
2. GeeksforGeeks. File Handling in Java. <https://www.geeksforgeeks.org/file-handling-in-java/>
3. W3Schools. Java Tutorial. <https://www.w3schools.com/java/>
4. Oracle. The Java Tutorials — Scanner and Basic I/O. <https://docs.oracle.com/javase/tutorial/>
5. Mermaid.js Documentation. Diagramming and charting tool. <https://mermaid.js.org/>